

Python Competitive Programming Reference Book

Arteaga Adán

Algorithms Sheet - Python Edition

Clean, contest-ready, interview-friendly snippets

Python version

Python version for competitive programming

Contents

1	Python Contest Setup	1
1.1	Fast Input	1
1.2	Read All Tokens	1
1.3	Useful Imports	1
1.4	Recursion Limit	1
1.5	Infinity and Constants	2
1.6	Python Contest Notes	2
2	Complexities and Python Structures	3
2.1	Common Time Complexities	3
2.2	Python Data Structures	3
2.3	Standard Constraints	3
2.4	Important Algorithms	4
3	Input, Strings, and Parsing Basics	5
3.1	Read a Full Line	5
3.2	Split a Line	5
3.3	Character and Word Tools	5
3.4	Sorting Strings	5
4	Binary Search	6
4.1	Basic Binary Search	6
4.2	Binary Search on Answer	6
4.3	bisect Left and Right	6
4.4	Last True / First True Pattern	7
5	Math and Number Theory	8
5.1	GCD and LCM	8
5.2	Manual Euclidean GCD	8
5.3	Classic Sieve + Factorization	8
5.4	Linear SPF Sieve + Factorization	9
5.5	Euler's Totient Function	9
5.6	Totient Sieve	9
5.7	Notable Sums	10
6	Modular Arithmetic and Combinatorics	11
6.1	Binary Exponentiation	11

6.2	Modular Inverse	11
6.3	Modulo Properties	11
6.4	Modular Factorials	11
6.5	Modular Combinations	12
6.6	Precomputed Inverse Factorials	12
7	Prefix Sums and Difference Arrays	13
7.1	Prefix Sum 1D	13
7.2	Prefix XOR	13
7.3	Prefix Sum 2D	13
7.4	Difference Array	13
8	Bit Manipulation	14
8.1	Basic Operations	14
8.2	Least Significant Bit	14
8.3	Python Bit Built-ins	14
8.4	All Bits Set of Size n	14
8.5	Check Power of Two	14
8.6	XOR Properties	14
8.7	Iterating All Subsets	15
8.8	Iterating Submasks	15
8.9	Subsets of Size K - Gosper's Hack	15
9	Core Graph Algorithms	16
9.1	Adjacency List	16
9.2	DFS Recursive	16
9.3	DFS Iterative	16
9.4	BFS	17
9.5	BFS Distances	17
9.6	Grid Directions	17
9.7	Graph Properties	18
10	Graph Coloring and Cycle Detection	19
10.1	Bipartite Coloring	19
10.2	Greedy Graph Coloring	19
10.3	Cycle Detection - Undirected Graph	19
10.4	Cycle Detection - Directed Graph	20
11	Shortest Paths and DSU	21

11.1	Dijkstra	21
11.2	Disjoint Set Union	21
11.3	Floyd-Warshall	22
12	String Matching Algorithms	23
12.1	KMP Prefix Function	23
12.2	KMP Pattern Matching	23
12.3	Z-Function	23
12.4	Z-Function Pattern Matching	24
12.5	Manacher - Odd Palindromes	24
12.6	Manacher - Even Palindromes	24
13	Dynamic Programming and Sequences	25
13.1	LIS - Longest Increasing Subsequence	25
13.2	Maximum Subarray Sum - Kadane	25
13.3	Basic 1D DP Pattern	25
13.4	Bitmask DP Skeleton	26
14	Advanced Data Structures	27
14.1	Fenwick Tree - BIT	27
14.2	Segment Tree - Range Sum	27
14.3	Coordinate Compression	28
14.4	Order Statistics with Fenwick	28
14.5	Heap Queue	28
15	General Techniques and Tricks	29
15.1	Two Pointers	29
15.2	Sliding Window Sum	29
15.3	Frequency Map	29
15.4	Sorting with Keys	29
15.5	Output Fast	29
15.6	Memory Tricks	30

1 Python Contest Setup

1.1 Fast Input

Use this when input is large. It is usually faster than repeated `input()`.

```
import sys
input = sys.stdin.readline

n = int(input())
a = list(map(int, input().split()))
```

1.2 Read All Tokens

Best for very large input with many integers.

```
import sys

data = sys.stdin.buffer.read().split()
it = iter(data)

n = int(next(it))
a = [int(next(it)) for _ in range(n)]
```

1.3 Useful Imports

```
import sys
from math import gcd, isqrt, inf
from collections import deque, defaultdict, Counter
from bisect import bisect_left, bisect_right
from heapq import heappush, heappop
from itertools import accumulate, permutations, combinations
```

1.4 Recursion Limit

For deep DFS, increase the recursion limit. If recursion still risks stack issues, use iterative DFS.

```
import sys
sys.setrecursionlimit(1_000_000)
```

1.5 Infinity and Constants

```
INF = 10**30
MOD = 998244353
# Common alternative:
MOD2 = 10**9 + 7
```

1.6 Python Contest Notes

- Prefer `sys.stdin.readline` or `buffer.read().split()` for heavy input.
- Use `deque` for queues, not `list.pop(0)`.
- Use `heapq` for priority queues.
- Use `bisect` for binary search on sorted arrays.
- Use `dict`, `set`, and `Counter` heavily.
- Prefer iterative DFS for very deep graphs.

2 Complexities and Python Structures

2.1 Common Time Complexities

- $O(1)$ - constant: index access, hash lookup average case.
- $O(\log n)$ - logarithmic: binary search, heap push/pop.
- $O(\sqrt{n})$ - square root: divisors, simple primality checks.
- $O(n)$ - linear: one pass over an array.
- $O(n \log n)$ - sorting, many greedy solutions.
- $O(n^2)$ - nested loops, some dynamic programming.
- $O(2^n)$ - subsets and bitmask DP.
- $O(n!)$ - permutations.

2.2 Python Data Structures

- `list`: append/pop back $O(1)$ amortized, insert/delete middle $O(n)$.
- `dict/set`: insert/find average $O(1)$, worst $O(n)$.
- `deque`: push/pop both ends $O(1)$.
- `heapq`: push/pop $O(\log n)$, min element $O(1)$.
- `bisect`: search in sorted list $O(\log n)$, insertion still $O(n)$.
- `Counter`: frequency map.

2.3 Standard Constraints

- Python is slower than compiled languages, so constants matter.
- $n = 10^5$: usually $O(n)$ or $O(n \log n)$.
- $n = 500$: $O(n^3)$ may pass depending on constants.
- $n = 20$: $O(2^n \cdot n)$ is common.
- 10^7 simple operations can be risky but possible with optimized code.

2.4 Important Algorithms

- GCD: $O(\log(\min(a, b)))$
- Binary exponentiation: $O(\log \exp)$
- Sieve of Eratosthenes: $O(n \log \log n)$
- Prime factorization by trial division: $O(\sqrt{n})$
- DFS/BFS: $O(V + E)$
- Dijkstra: $O(E \log V)$
- Floyd-Warshall: $O(V^3)$
- DSU: $O(\alpha(n))$ amortized
- Segment Tree query/update: $O(\log n)$
- Fenwick Tree query/update: $O(\log n)$
- KMP, Z-function, Manacher: $O(n)$

3 Input, Strings, and Parsing Basics

3.1 Read a Full Line

Equivalent to reading a whole line including spaces.

```
s = input().rstrip('\n')
```

3.2 Split a Line

Equivalent to tokenizing a string by whitespace.

```
s = "10 20 30"
a, b, c = map(int, s.split())
```

3.3 Character and Word Tools

```
from collections import Counter

s = "abacaba"
freq = Counter(s)

words = "one two three".split()
joined = "-".join(words)
```

3.4 Sorting Strings

```
s = "dcba"
t = ''.join(sorted(s))

arr = ["aaa", "b", "cc"]
arr.sort(key=len)
```

4 Binary Search

4.1 Basic Binary Search

```
l, r = 0, n - 1
ans = -1

while l <= r:
    mid = (l + r) // 2
    if a[mid] == x:
        ans = mid
        break
    if a[mid] < x:
        l = mid + 1
    else:
        r = mid - 1
```

4.2 Binary Search on Answer

Find the minimum value satisfying a monotonic predicate.

```
l, r = 0, 10**18
ans = r

while l <= r:
    mid = (l + r) // 2
    if check(mid):
        ans = mid
        r = mid - 1
    else:
        l = mid + 1
```

4.3 bisect Left and Right

```
from bisect import bisect_left, bisect_right

idx = bisect_left(a, x) # first position with a[i] >= x
idx2 = bisect_right(a, x) # first position with a[i] > x

count_x = bisect_right(a, x) - bisect_left(a, x)
exists = bisect_left(a, x) < len(a) and a[bisect_left(a, x)] == x
```

4.4 Last True / First True Pattern

```
# first True over [lo, hi)
lo, hi = 0, n
while lo < hi:
    mid = (lo + hi) // 2
    if ok(mid):
        hi = mid
    else:
        lo = mid + 1
ans = lo
```

5 Math and Number Theory

5.1 GCD and LCM

```
from math import gcd

def lcm(a, b):
    if a == 0 or b == 0:
        return 0
    return abs(a // gcd(a, b) * b)
```

Properties:

- $\gcd(a, b) \cdot \text{lcm}(a, b) = |a \cdot b|$
- $\gcd(a, b) = \gcd(b, a \bmod b)$
- $\gcd(a, b, c) = \gcd(a, \gcd(b, c))$

5.2 Manual Euclidean GCD

```
def gcd_manual(a, b):
    while b:
        a, b = b, a % b
    return abs(a)
```

5.3 Classic Sieve + Factorization

```
def sieve(n):
    is_prime = [True] * (n + 1)
    is_prime[0:2] = [False, False]
    primes = []
    for i in range(2, n + 1):
        if is_prime[i]:
            primes.append(i)
            if i * i <= n:
                for j in range(i * i, n + 1, i):
                    is_prime[j] = False
    return is_prime, primes

def factorize(x, primes):
    f = []
    for p in primes:
        if p * p > x:
            break
        while x % p == 0:
            f.append(p)
            x //= p
    if x > 1:
        f.append(x)
    return f
```

5.4 Linear SPF Sieve + Factorization

```
def linear_sieve_spf(n):
    spf = [0] * (n + 1)
    primes = []
    for i in range(2, n + 1):
        if spf[i] == 0:
            spf[i] = i
            primes.append(i)
        for p in primes:
            if p > spf[i] or i * p > n:
                break
            spf[i * p] = p
    return spf, primes

def factorize_spf(x, spf):
    f = []
    while x > 1:
        f.append(spf[x])
        x //= spf[x]
    return f
```

5.5 Euler's Totient Function

```
def phi_single(n):
    res = n
    p = 2
    while p * p <= n:
        if n % p == 0:
            while n % p == 0:
                n //= p
            res -= res // p
        p += 1
    if n > 1:
        res -= res // n
    return res
```

5.6 Totient Sieve

```
def phi_sieve(n):
    phi = list(range(n + 1))
    for i in range(2, n + 1):
        if phi[i] == i: # prime
            for j in range(i, n + 1, i):
                phi[j] -= phi[j] // i
    return phi
```

5.7 Notable Sums

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$$

```
sum_1_to_n = n * (n + 1) // 2
sum_squares = n * (n + 1) * (2 * n + 1) // 6
```

6 Modular Arithmetic and Combinatorics

6.1 Binary Exponentiation

Python already has optimized modular exponentiation through `pow(a,b,mod)`.

```
def binpow(a, b, mod):
    res = 1
    a %= mod
    while b:
        if b & 1:
            res = res * a % mod
        a = a * a % mod
        b >>= 1
    return res

ans = pow(a, b, MOD)
```

6.2 Modular Inverse

Works when MOD is prime and a is not divisible by MOD.

```
def mod_inverse(a, mod):
    return pow(a, mod - 2, mod)
```

6.3 Modulo Properties

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a - b) \bmod m = ((a \bmod m) - (b \bmod m)) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m)(b \bmod m)) \bmod m$$

In Python, `x % MOD` already returns a non-negative result for positive MOD.

6.4 Modular Factorials

```
MOD = 998244353
MAXN = 10**6 + 5
fact = [1] * MAXN

for i in range(1, MAXN):
    fact[i] = fact[i - 1] * i % MOD
```

6.5 Modular Combinations

```
def nCr(n, r):  
    if r < 0 or r > n:  
        return 0  
    return fact[n] * pow(fact[r], MOD - 2, MOD) % MOD * pow(fact[n - r], MOD - 2, MOD) % MOD
```

$$C(n, r) = \frac{n!}{r!(n-r)!} \quad P(n, r) = \frac{n!}{(n-r)!}$$

6.6 Precomputed Inverse Factorials

Faster when there are many queries.

```
fact = [1] * (MAXN + 1)  
invfact = [1] * (MAXN + 1)  
  
for i in range(1, MAXN + 1):  
    fact[i] = fact[i - 1] * i % MOD  
  
invfact[MAXN] = pow(fact[MAXN], MOD - 2, MOD)  
for i in range(MAXN, 0, -1):  
    invfact[i - 1] = invfact[i] * i % MOD  
  
def comb(n, r):  
    if r < 0 or r > n:  
        return 0  
    return fact[n] * invfact[r] % MOD * invfact[n - r] % MOD
```


7 Prefix Sums and Difference Arrays

7.1 Prefix Sum 1D

```
pref = [0] * (n + 1)
for i in range(n):
    pref[i + 1] = pref[i] + a[i]

def range_sum(l, r): # inclusive, 0-indexed
    return pref[r + 1] - pref[l]
```

7.2 Prefix XOR

```
px = [0] * (n + 1)
for i in range(n):
    px[i + 1] = px[i] ^ a[i]

def range_xor(l, r):
    return px[r + 1] ^ px[l]
```

7.3 Prefix Sum 2D

```
p = [[0] * (m + 1) for _ in range(n + 1)]

for i in range(1, n + 1):
    for j in range(1, m + 1):
        p[i][j] = a[i - 1][j - 1] + p[i - 1][j] + p[i][j - 1] - p[i - 1][j - 1]

def rect_sum(x1, y1, x2, y2):
    # inclusive, 0-indexed
    x1 += 1; y1 += 1; x2 += 1; y2 += 1
    return p[x2][y2] - p[x1 - 1][y2] - p[x2][y1 - 1] + p[x1 - 1][y1 - 1]
```

7.4 Difference Array

Range update $[l, r]$ with value v .

```
d = [0] * (n + 1)

def add_range(l, r, v):
    d[l] += v
    if r + 1 < len(d):
        d[r + 1] -= v

cur = 0
for i in range(n):
    cur += d[i]
    a[i] += cur
```

8 Bit Manipulation

8.1 Basic Operations

```
mask | (1 << i)      # set bit i
mask & (1 << i)      # check bit i
mask ^ (1 << i)      # toggle bit i
mask & ~(1 << i)     # clear bit i
```

8.2 Least Significant Bit

```
mask & -mask         # isolate LSB
mask & (mask - 1)    # remove LSB
```

8.3 Python Bit Built-ins

```
x.bit_count()        # number of 1 bits
x.bit_length()       # bits needed to represent x
x.bit_length() - 1   # floor(log2(x)), if x > 0
```

8.4 All Bits Set of Size n

```
full = (1 << n) - 1
```

8.5 Check Power of Two

```
def is_power_of_two(n):
    return n > 0 and (n & (n - 1)) == 0
```

8.6 XOR Properties

- $a \oplus a = 0$
- $a \oplus 0 = a$
- If $a \oplus b = c$, then $a \oplus c = b$.

8.7 Iterating All Subsets

```
for mask in range(1 << n):
    for j in range(n):
        if mask & (1 << j):
            pass # j is in subset
```

8.8 Iterating Submasks

```
sub = mask
while sub:
    # sub is a submask of mask
    sub = (sub - 1) & mask
# excludes empty submask 0
```

8.9 Subsets of Size K - Gosper's Hack

```
set_mask = (1 << k) - 1
while set_mask < (1 << n):
    # process set_mask
    c = set_mask & -set_mask
    r = set_mask + c
    set_mask = (((r ^ set_mask) >> 2) // c) | r
```

9 Core Graph Algorithms

9.1 Adjacency List

```
adj = [[] for _ in range(n)]

# undirected edge
adj[u].append(v)
adj[v].append(u)

# directed edge
adj[u].append(v)
```

9.2 DFS Recursive

```
import sys
sys.setrecursionlimit(1_000_000)

vis = [False] * n

def dfs(u):
    vis[u] = True
    for v in adj[u]:
        if not vis[v]:
            dfs(v)
```

9.3 DFS Iterative

Safer when recursion depth is dangerous.

```
def dfs_iter(start):
    vis = [False] * n
    stack = [start]
    vis[start] = True
    while stack:
        u = stack.pop()
        for v in adj[u]:
            if not vis[v]:
                vis[v] = True
                stack.append(v)
    return vis
```

9.4 BFS

```
from collections import deque

def bfs(s):
    vis = [False] * n
    q = deque([s])
    vis[s] = True
    while q:
        u = q.popleft()
        for v in adj[u]:
            if not vis[v]:
                vis[v] = True
                q.append(v)
    return vis
```

9.5 BFS Distances

```
from collections import deque

def bfs_dist(s):
    dist = [-1] * n
    q = deque([s])
    dist[s] = 0
    while q:
        u = q.popleft()
        for v in adj[u]:
            if dist[v] == -1:
                dist[v] = dist[u] + 1
                q.append(v)
    return dist
```

9.6 Grid Directions

```
dx = [-1, 1, 0, 0]
dy = [0, 0, -1, 1]

for k in range(4):
    nx = x + dx[k]
    ny = y + dy[k]
    if 0 <= nx < R and 0 <= ny < C:
        pass
```

9.7 Graph Properties

- Tree edges: $n - 1$.
- Complete graph edges: $\frac{n(n-1)}{2}$.
- Handshaking lemma: $\sum \deg(v) = 2|E|$.

10 Graph Coloring and Cycle Detection

10.1 Bipartite Coloring

2-color an undirected graph. If impossible, the graph is not bipartite.

```
from collections import deque

def bipartite_color(adj):
    n = len(adj)
    color = [-1] * n
    for s in range(n):
        if color[s] != -1:
            continue
        color[s] = 0
        q = deque([s])
        while q:
            u = q.popleft()
            for v in adj[u]:
                if color[v] == -1:
                    color[v] = color[u] ^ 1
                    q.append(v)
                elif color[v] == color[u]:
                    return False, color
    return True, color
```

10.2 Greedy Graph Coloring

Assigns a valid coloring, not necessarily minimum.

```
def greedy_coloring(adj):
    n = len(adj)
    color = [-1] * n
    for u in range(n):
        used = {color[v] for v in adj[u] if color[v] != -1}
        c = 0
        while c in used:
            c += 1
        color[u] = c
    return color
```

10.3 Cycle Detection - Undirected Graph

```
def has_cycle_undirected(adj):
    n = len(adj)
    vis = [False] * n

    def dfs(u, parent):
        vis[u] = True
        for v in adj[u]:
            if not vis[v]:
                if dfs(v, u):
                    return True
            elif v != parent:
                return True
        return False

    for i in range(n):
        if not vis[i] and dfs(i, -1):
            return True
    return False
```

10.4 Cycle Detection - Directed Graph

Use states: 0 = unvisited, 1 = visiting, 2 = done.

```
def has_cycle_directed(adj):
    n = len(adj)
    state = [0] * n

    def dfs(u):
        state[u] = 1
        for v in adj[u]:
            if state[v] == 1:
                return True
            if state[v] == 0 and dfs(v):
                return True
        state[u] = 2
        return False

    for i in range(n):
        if state[i] == 0 and dfs(i):
            return True
    return False
```


11 Shortest Paths and DSU

11.1 Dijkstra

Graph format: `adj[u] = [(v, w), ...]`.

```
from heapq import heappush, heappop

INF = 10**30

def dijkstra(adj, s):
    n = len(adj)
    dist = [INF] * n
    dist[s] = 0
    pq = [(0, s)]

    while pq:
        d, u = heappop(pq)
        if d != dist[u]:
            continue
        for v, w in adj[u]:
            nd = d + w
            if nd < dist[v]:
                dist[v] = nd
                heappush(pq, (nd, v))
    return dist
```

11.2 Disjoint Set Union

```
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.sz = [1] * n

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def unite(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.sz[ra] < self.sz[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.sz[ra] += self.sz[rb]
        return True
```

11.3 Floyd-Warshall

All-pairs shortest paths. Use only for small n .

```
for k in range(n):
    for i in range(n):
        dik = dist[i][k]
        for j in range(n):
            nd = dik + dist[k][j]
            if nd < dist[i][j]:
                dist[i][j] = nd
```

12 String Matching Algorithms

12.1 KMP Prefix Function

```
def prefix_function(s):
    n = len(s)
    pi = [0] * n
    for i in range(1, n):
        j = pi[i - 1]
        while j > 0 and s[i] != s[j]:
            j = pi[j - 1]
        if s[i] == s[j]:
            j += 1
        pi[i] = j
    return pi
```

12.2 KMP Pattern Matching

Find all occurrences of pattern p in text t .

```
def kmp_search(t, p):
    s = p + '#' + t
    pi = prefix_function(s)
    ans = []
    m = len(p)
    for i in range(m + 1, len(s)):
        if pi[i] == m:
            ans.append(i - 2 * m)
    return ans
```

12.3 Z-Function

```
def z_function(s):
    n = len(s)
    z = [0] * n
    l = r = 0
    for i in range(1, n):
        if i <= r:
            z[i] = min(r - i + 1, z[i - l])
        while i + z[i] < n and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] - 1 > r:
            l, r = i, i + z[i] - 1
    return z
```

12.4 Z-Function Pattern Matching

```
def z_search(t, p):
    s = p + '#' + t
    z = z_function(s)
    m = len(p)
    return [i - m - 1 for i in range(m + 1, len(s)) if z[i] >= m]
```

12.5 Manacher - Odd Palindromes

$d1[i]$ = radius of odd palindrome centered at i .

```
def manacher_odd(s):
    n = len(s)
    d1 = [0] * n
    l, r = 0, -1
    for i in range(n):
        k = 1 if i > r else min(d1[l + r - i], r - i + 1)
        while 0 <= i - k and i + k < n and s[i - k] == s[i + k]:
            k += 1
        d1[i] = k
        if i + k - 1 > r:
            l, r = i - k + 1, i + k - 1
    return d1
```

12.6 Manacher - Even Palindromes

$d2[i]$ = radius of even palindrome centered between $i-1$ and i .

```
def manacher_even(s):
    n = len(s)
    d2 = [0] * n
    l, r = 0, -1
    for i in range(n):
        k = 0 if i > r else min(d2[l + r - i + 1], r - i + 1)
        while 0 <= i - k - 1 and i + k < n and s[i - k - 1] == s[i + k]:
            k += 1
        d2[i] = k
        if i + k - 1 > r:
            l, r = i - k, i + k - 1
    return d2
```

13 Dynamic Programming and Sequences

13.1 LIS - Longest Increasing Subsequence

$O(n \log n)$ length only.

```
from bisect import bisect_left, bisect_right

def lis_length(a):
    lis = []
    for x in a:
        i = bisect_left(lis, x)
        if i == len(lis):
            lis.append(x)
        else:
            lis[i] = x
    return len(lis)

# For longest non-decreasing subsequence, use bisect_right.
```

13.2 Maximum Subarray Sum - Kadane

Works even if all numbers are negative.

```
def kadane(a):
    best = -10**30
    cur = 0
    for x in a:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

13.3 Basic 1D DP Pattern

```
dp = [0] * (n + 1)
dp[0] = 1

for i in range(1, n + 1):
    # transition here
    dp[i] = dp[i - 1]
```

13.4 Bitmask DP Skeleton

```
INF = 10**30
dp = [INF] * (1 << n)
dp[0] = 0

for mask in range(1 << n):
    for i in range(n):
        if not (mask & (1 << i)):
            new_mask = mask | (1 << i)
            dp[new_mask] = min(dp[new_mask], dp[mask] + cost[i])
```

14 Advanced Data Structures

14.1 Fenwick Tree - BIT

1-indexed structure for prefix sums.

```
class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0] * (n + 1)

    def add(self, idx, val):
        while idx <= self.n:
            self.bit[idx] += val
            idx += idx & -idx

    def sum(self, idx):
        res = 0
        while idx > 0:
            res += self.bit[idx]
            idx -= idx & -idx
        return res

    def range_sum(self, l, r):
        return self.sum(r) - self.sum(l - 1)
```

14.2 Segment Tree - Range Sum

Iterative version: compact and fast in Python.

```
class SegTree:
    def __init__(self, a):
        self.n = len(a)
        self.t = [0] * (2 * self.n)
        self.t[self.n:] = a[:]
        for i in range(self.n - 1, 0, -1):
            self.t[i] = self.t[i << 1] + self.t[i << 1 | 1]

    def update(self, pos, val):
        pos += self.n
        self.t[pos] = val
        pos >>= 1
        while pos:
            self.t[pos] = self.t[pos << 1] + self.t[pos << 1 | 1]
            pos >>= 1

    def query(self, l, r): # inclusive [l, r]
        l += self.n
        r += self.n + 1
        res = 0
        while l < r:
            if l & 1:
                res += self.t[l]
                l += 1
            if r & 1:
                r -= 1
                res += self.t[r]
            l >>= 1
            r >>= 1
        return res
```

14.3 Coordinate Compression

```
vals = sorted(set(a))
comp = {x: i for i, x in enumerate(vals)}
compressed = [comp[x] for x in a]
```

14.4 Order Statistics with Fenwick

Useful when values are compressed to $1..m$.

```
def kth(bit, k):
    # returns smallest idx with prefix sum >= k
    idx = 0
    step = 1 << (bit.n.bit_length() - 1)
    while step:
        nxt = idx + step
        if nxt <= bit.n and bit.bit[nxt] < k:
            idx = nxt
            k -= bit.bit[nxt]
        step >>= 1
    return idx + 1
```

14.5 Heap Queue

```
from heapq import heappush, heappop
```

```
pq = []
heappush(pq, (dist, node))
d, u = heappop(pq)
```

```
# max-heap trick
heappush(pq, -x)
x = -heappop(pq)
```


15 General Techniques and Tricks

15.1 Two Pointers

```
l = 0
for r in range(n):
    # add a[r]
    while bad(l, r):
        # remove a[l]
        l += 1
    # current window is [l, r]
```

15.2 Sliding Window Sum

```
cur = sum(a[:k])
best = cur
for r in range(k, n):
    cur += a[r] - a[r - k]
    best = max(best, cur)
```

15.3 Frequency Map

```
from collections import defaultdict, Counter

freq = defaultdict(int)
for x in a:
    freq[x] += 1

cnt = Counter(a)
```

15.4 Sorting with Keys

```
points.sort() # lexicographic
points.sort(key=lambda p: p[1]) # by y
points.sort(key=lambda p: (p[0], -p[1]))
```

15.5 Output Fast

```
import sys

ans = ["YES", "NO", "YES"]
sys.stdout.write("\n".join(ans))
```

15.6 Memory Tricks

- Avoid huge lists of lists if a sparse structure works.
- For grids, store characters as strings when read-only.
- Use arrays of integers instead of dictionaries when indices are dense.
- Use `bytearray(n)` for boolean-like arrays when memory is tight.